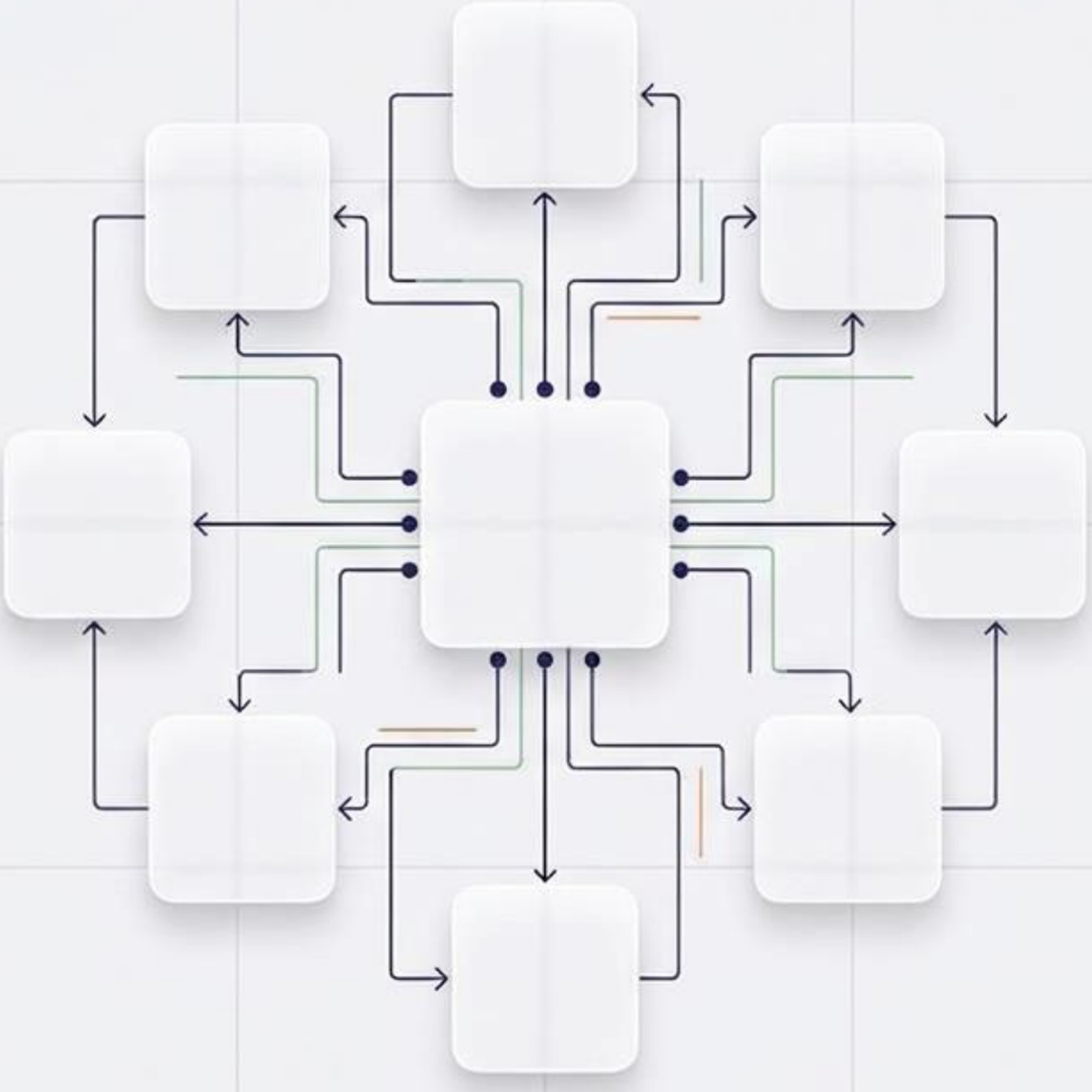
IntraNet

Dashboard de Comunicação Interna: Modelagem de Dados e PRD

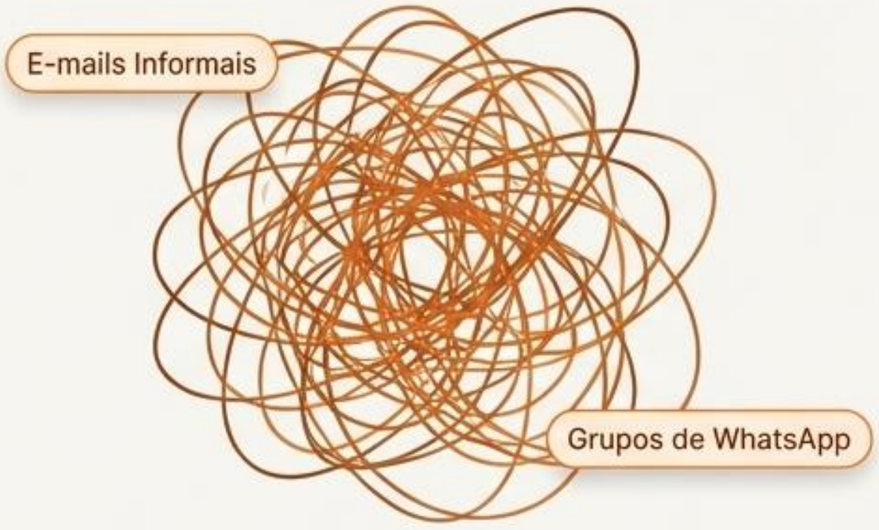
Alunos: João David Salomão, Lucas da Silva Santos, Pollyana de Castro Rodrigues
Disciplina: Modelagem de Dados
Ano: 2026



NotebookLM

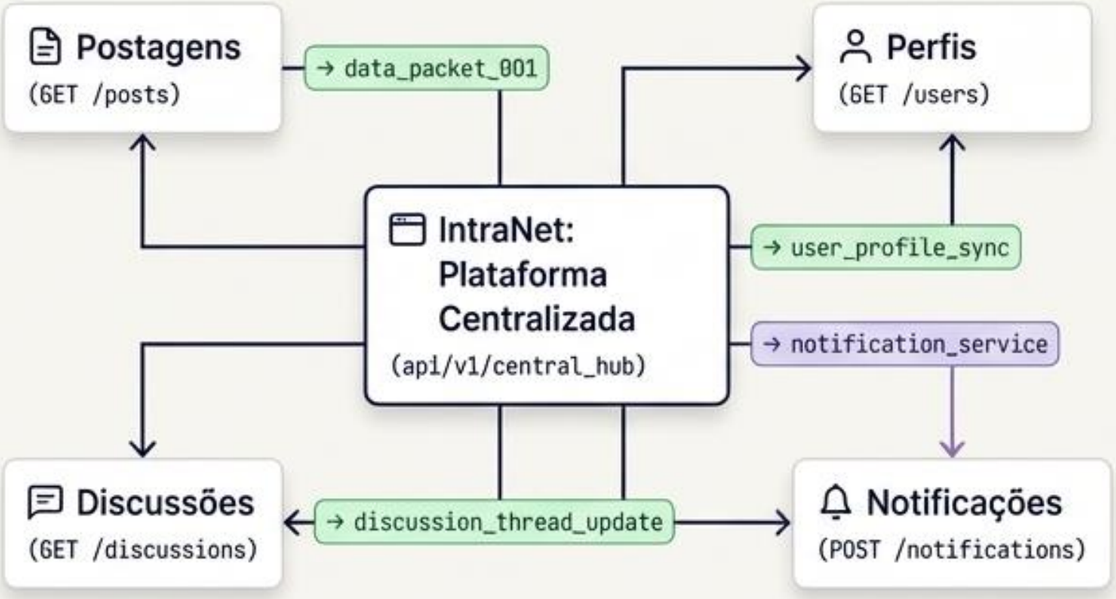
O Fim da Fragmentação de Dados

O Problema



Contexto: A comunicação interna atual depende de canais informais, resultando em perda de informações e falta de rastreabilidade.

A Solução



A Visão IntraNet: Um dashboard web moderno que centraliza postagens, perfis de colaboradores e discussões. Uma interface única para conectar a equipe, substituindo o caos por uma estrutura de dados previsível e acessível.

NotebookLM

Prompt Utilizado para Geração das User Stories

Conforme solicitado, registra-se abaixo o prompt utilizado no Claude para auxiliar na elaboração das histórias de usuário deste documento:

Estou desenvolvendo um protótipo de um Dashboard de Comunicação Interna para uma startup, como trabalho da disciplina de Modelagem de Dados. O sistema consome dados da API pública JSONPlaceholder, que disponibliza os endpoints GET /users (lista de colaboradores), GET /posts?userId={id} (postagens por colaborador) e GET /comments?postId={id} (comentários por postagem).

O produto se chama IntraNet e tem como objetivo centralizar postagens, perfis de colaboradores e comentários em uma interface web moderna, substituindo o uso informal de e-mails e grupos de WhatsApp para comunicação interna.

Com base nesse contexto, gere 5 User Stories no formato ágil padrão: 'Como [perfil do usuário], desejo [ação ou funcionalidade], para que [benefício ou objetivo].' Os perfis de usuário possíveis são: Funcionário, Gestor e Usuário genérico. Cada história deve estar diretamente relacionada a uma funcionalidade real que o entlira tan deve ataronimar o sistema entregará, e ao menos uma delas deve tratar de tolerância a falhas de rede ou indisponibillidade da API.

Gere Também o Documento PRD completo contendo as especificações necessárias para o Projeto ser totalmente viável.

NotebookLM
NotebookLM

A Jornada do Usuário (User Stories)

US-01

Perfil: Funcionário

Ação: Ver lista de colaboradores.

Benefício: Encontrar contatos rapidamente.

`GET /users`

US-02

Perfil: Funcionário

Ação: Ver postagens recentes de um colega.

Benefício: Atualização de projetos.

`GET /posts?userId={id}`

US-03

Perfil: Gestor

Ação: Visualizar comentários.

Benefício: Acompanhar discussões da equipe.

`GET /comments?postId={id}`

US-04

Perfil: Funcionário

Ação: Buscar colaborador.

Benefício: Acesso rápido sem rolar lista.

`Filtro Local`

US-05

Perfil: Usuário

Ação: Receber alerta visual de erro/lentidão.

Benefício: Feedback de status do sistema.

`AbortController + Toast`

NotebookLM

A Superfície do Produto: Requisitos Funcionais

Recuperação de Dados (Core API)

RF-01: Listar todos os colaboradores (`GET /users`).
RF-02: Carregar postagens ao selecionar colaborador (`GET /posts`).
RF-03: Carregar comentários atrelados à postagem (`GET /comments`).

Interatividade & Busca

RF-04: Filtro de busca em tempo real por nome ou e-mail.

Experiência do Usuário (UX)

RF-05: Estados de carregamento (skeleton) durante requisições.
RF-06: Mensagens de erro visíveis com opção de nova tentativa.

Otimização de Interface

RF-07: Cache local de comentários para evitar requisições duplicadas.

NotebookLM

A Fundação Invisível: Requisitos Não-Funcionais

NotebookLM

Decisão Arquitetural: Busca na Rede vs. Memória

Busca Padrão API

Método
Nova requisição `GET /users?search=query`

Latência
200ms a 3000ms (dependente da rede)

Custo de Servidor
Alto (Múltiplas chamadas por caractere)

Resiliência
Falha se a rede cair durante a digitação

Abordagem IntraNet em Memória - RNF-04

Método
`Filtro local de array (Array.filter)`

Latência
< 5ms (Imediato)

Custo de Servidor
Zero (Processamento no cliente)

Resiliência
Funciona **100% offline** após o carregamento inicial

NotebookLM

Proteção Ativa: Gerenciamento de Timeout (US-05)

Anatomia de uma Falha Elegante

0s

Requisição Iniciada

Usuário clica em ver comentários (Fetch API + AbortController ativado).

4s

Rede Lenta/Indisponível

Servidor JSONPlaceholder não responde.

8s

Timeout Alcançado (RNF-01)

AbortController envia sinal de cancelamento. Bloco catch captura a exceção (RNF-02).

+10ms

Resolução de Interface (RNF-03)

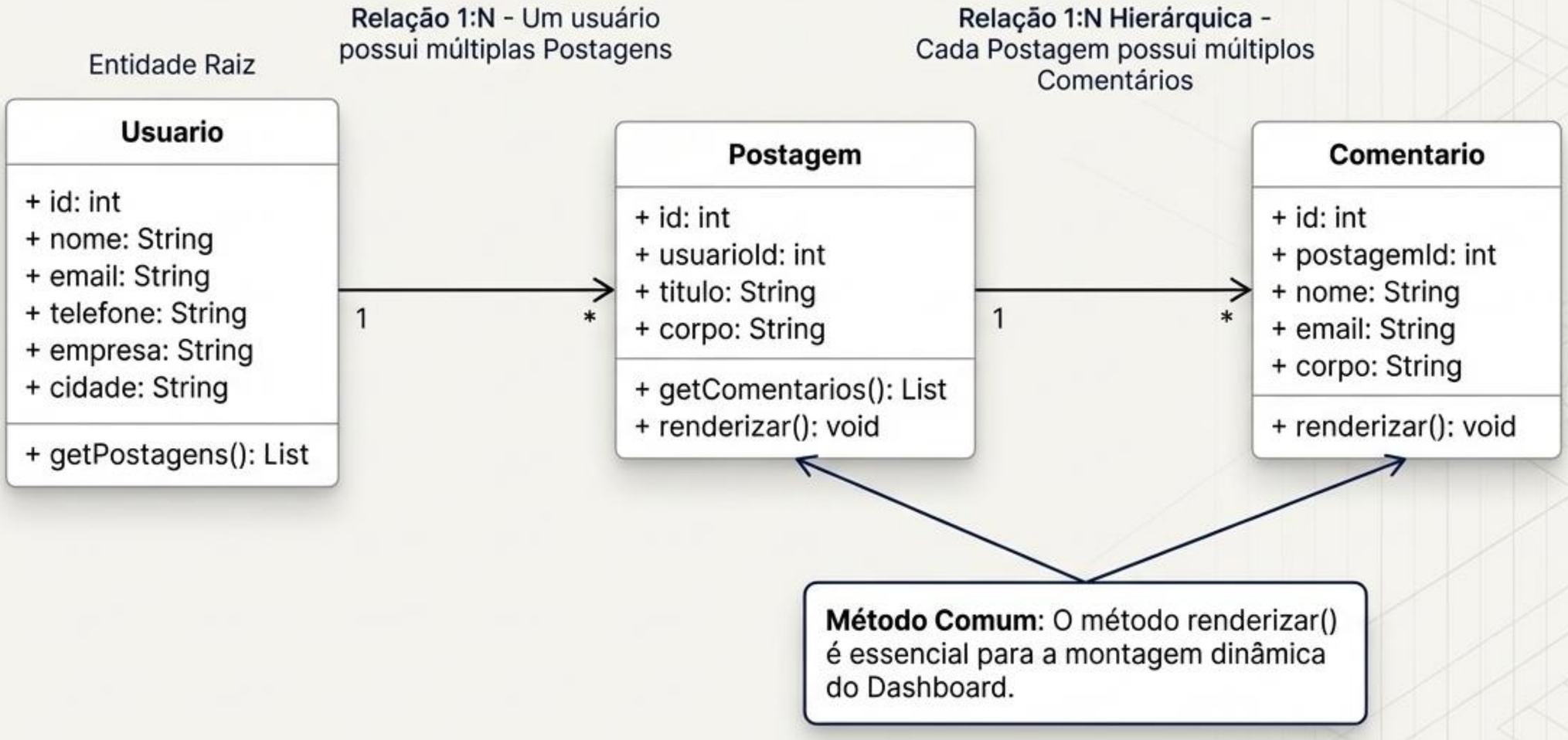
O sistema não trava. O estado de Loading é encerrado graciosamente.

Feedback Visual (US-05)

Não foi possível carregar os dados. Tentar novamente.

NotebookLM

O Núcleo de Dados: Diagrama de Classes UML



NotebookLM

Síntese Arquitetural: Rastreabilidade Total

Classe UML	Endpoint API	Requisito Técnico	User Story
Usuario	`GET /users`	Filtro Local em Memória (RNF-04)	Buscar colaborador (US-04)
Postagem	`GET /posts`	Tratamento Try/Catch (RNF-02)	Ver postagens recentes (US-02)
Comentario	Método `renderizar()`	Cache local (RF-07)	Visualizar comentários (US-03)

Cada atributo de dado mapeado possui um propósito funcional, garantindo um produto final coeso, performático e resiliente a falhas.

NotebookLM

Padrão MVC Aplicado ao Protótipo

MODEL

```
fetchWithTimeout()
fetchUsers()
fetchPosts()
fetchComments()
```

Timeout 8s · try/catch

VIEW

```
renderUserItem()
renderProfile()
renderPost()
renderComment()
```

CONTROLLER

```
init()
selectUser(userId)
toggleComments(postId)
filterUsers()
```

🌐 JSONPlaceholder REST API → GET /users · GET /posts?userId · GET /comments?postId

NotebookLM

Viabilidade Técnica

0
reais
Custo de infraestrutura no MVP

1
sprint
Tempo até protótipo funcional

3
endpoints
Recursos REST consumidos

- ✓ Sem necessidade de banco de dados no MVP — JSONPlaceholder simula o backend real.
- ✓ API RESTful pública e gratuita: validação da arquitetura com zero custo.
- ✓ Protótipo funcional demonstra viabilidade para investidores antes do backend proprietário.
- ✓ Arquitetura MVC facilita a substituição do Model (JSONPlaceholder → API própria) sem refatorar View/Controller.

NotebookLM

Matriz de Gerenciamento de Riscos

API fora do ar Alto	catch() exibe mensagem de erro amigável ao usuário. Botão "Tentar novamente" aciona nova requisição.	`try { data = await fetch(...) } catch(err) { renderError(err) }`
API lenta / sem resposta Alto	AbortController cancela a requisição após 8 segundos. Toast notifica o usuário sobre o timeout.	`setTimeout(() => controller.abort(), 8000)`
Mudança no formato do JSON Médio	Camada Model isolada: apenas o Model precisa ser atualizado. View e Controller permanecem intocados.	`Model.fetchUsers() → adapter layer`
Código HTTP 404 / 500 Médio	Verificação de res.ok após fetch. Throws Error com código HTTP para tratamento no catch.	`if (!res.ok) throw new Error(`HTTP ${res.status}`)`

NotebookLM

Roadmap Arquitetural e Próximos Passos

Fase 1 — MVP

- ✅ Protótipo com JSONPlaceholder
- ✅ Arquitetura MVC validada
- ✅ Tratamento de erros de rede

→

Fase 2 — Alpha

- → Backend Node.js + PostgreSQL
- → Autenticação JWT
- → Substituir Model para API própria

→

Fase 3 — Beta

- → Notificações em tempo real (WebSocket)
- → Upload de anexos
- → Painel administrativo

Obrigado!

NotebookLM